AI-ENG-110 · Spring 2026

Async Research Assistant

Topic 4 — AI Engineering Final Project

Team ExperimentMice · v1.0-final

Wikipedia | arXiv | Web Search

- Concurrent fetch — Wikipedia · arXiv · Web
- Exponential backoff retries + rate limiting
- Graceful degradation — partial results still synthesized
- TTL cache + optional PostgreSQL history

87%	Test Coverage
127	Tests Passing
8.82×	Live Speedup
5.16s	Live-Net Concurrent

github.com/abdurahmanligulnisa/research-assistant · Due May 23, 2026

Project Overview

A software-engineering wrapper around an async AI research pipeline

What the system does

01 User submits a research question via CLI or Streamlit Web UI

02 System concurrently fetches Wikipedia, arXiv & web sources

03 Results cached by (source, canonicalized_query) key with TTL

04 Duplicate URLs deduped — first-seen order preserved

05 LLM synthesizes a cited Markdown answer (thread-pool offload)

06 Session persisted to PostgreSQL; graceful fallback if absent

Key Design Decisions

ai.* always via AIService

No direct imports in business logic — enables mocking, retry, and logging

asyncio.to_thread for synthesis

LLM call is synchronous; offloaded so the event loop is never blocked

Graceful degradation

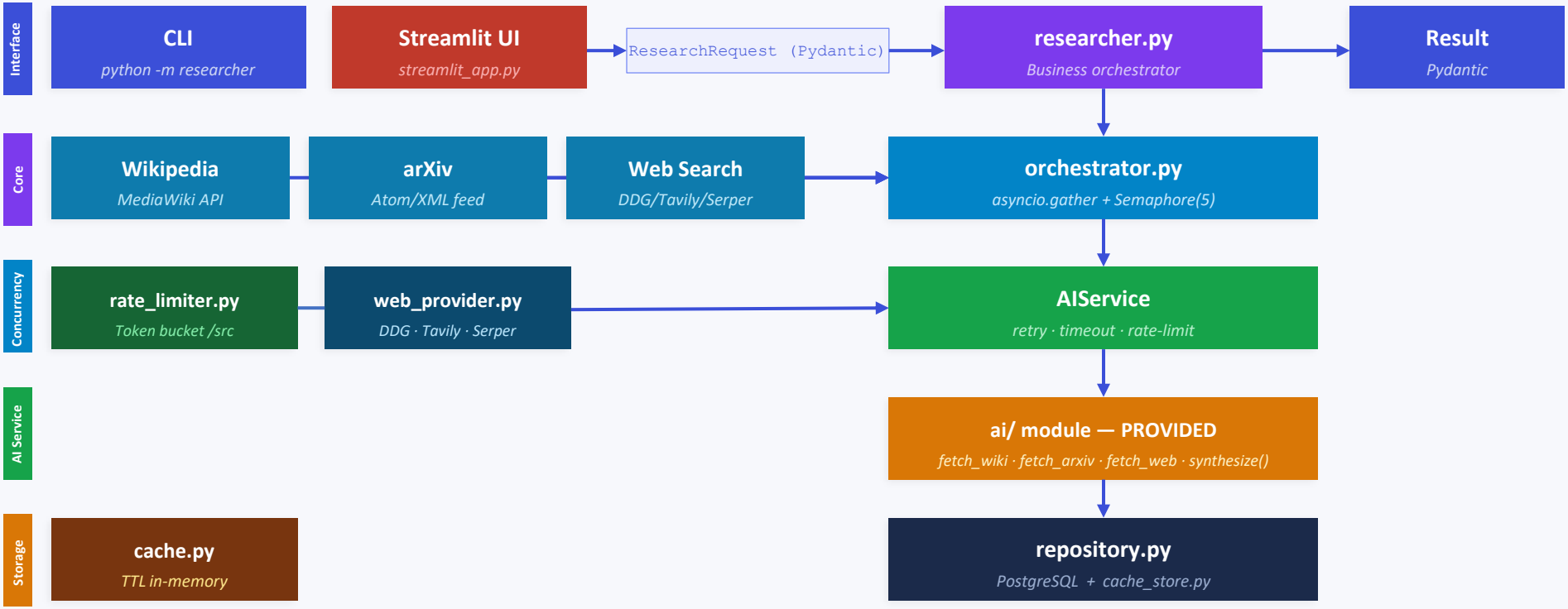
One failing source ≠ total failure; answer produced from remaining sources

Cache keyed by canonical query

Lowercased + whitespace-normalized key prevents redundant fetches

System Architecture

End-to-end pipeline — bounded concurrency, retry, graceful degradation



Technology Stack

Python 3.12 (Docker) / 3.13 (local) · fully pinned requirements.txt

Python 3.12 / 3.13

Language

Claude / GPT-4o / Gemini

LLM (pluggable)

httpx + asyncio

Async HTTP

tenacity

Exp. Backoff

Token-bucket limiter

Rate Limiting

Pydantic v2 + settings

Validation

Click

CLI

Streamlit

Web UI (bonus)

psycopg3 + PostgreSQL

Database

pytest + respx + httpx

Testing

ruff + mypy

Lint / Types

Docker + Compose

Container

Concurrent Source Orchestration

src/concurrency/orchestrator.py — asyncio.gather + Semaphore + per-source timeout

orchestrator.py

```
async def fetch_sources_concurrent(
    question: str,  sources: set[str] | None,
    max_concurrent: int | None = None,
) -> tuple[dict[str, list], list[str]]:

    limit = max_concurrent or settings.max_concurrent
    sem   = asyncio.Semaphore(limit)  # default=5

    async def _guarded(src_name: str):
        async with sem:
            return await _fetch_one(src_name, question,
                                    client=client)

    tasks   = [_guarded(s) for s in sorted(active)]
    results = await asyncio.gather(
        *tasks, return_exceptions=True  # partial ok
    )
```

Bounded Concurrency

Semaphore(MAX_CONCURRENT=5) prevents API flooding and rate-limit hits from parallel callers.

Graceful Degradation

return_exceptions=True — one failed source never aborts the run; partial results are synthesized.

Per-Source Timeout

asyncio.wait_for() enforces a hard wall-clock budget per source (wiki 4×, web 6× per-attempt).

Source Filtering

--sources wiki,arxiv limits active_set; missing sources skip gracefully with no error raised.

Retry & Rate-Limit Strategy

src/services/ai_service.py — tenacity decorator · RateLimitError · token-bucket per source

ai_service.py

```python
class RateLimitError(ProviderError):
    # Carries Retry-After hint parsed from header
    def __init__(self, msg, retry_after=None):
        super().__init__(msg)
        self.retry_after = retry_after

def _make_retry(max_attempts=4):
    def _before_sleep(retry_state):
        exc = retry_state.outcome.exception()
        if isinstance(exc, RateLimitError) and \
                exc.retry_after is not None:
            # honour server Retry-After header exactly
            retry_state.next_action.sleep = exc.retry_after
    return retry(
        wait=wait_exponential(min=1, max=30),
        stop=stop_after_attempt(max_attempts),
        before_sleep=_before_sleep, reraise=True)
```

Retried (_TRANSIENT)

- `ConnectionError`
- `TimeoutError`
- `OSError`
- `ProviderError (+ RateLimitError)`
- `RateLimitExceeded (token-bucket)`

① TokenBucketRateLimiter

Requests / second per source — auto-applied before every fetch

10 req/s	2 req/s	1 req/s
Wikipedia	arXiv	Web

② LlmTpmLimiter (NEW) — tokens-per-minute ceiling

`claude-sonnet: 40 000 · gpt-4o-mini: 200 000 · gemini-flash: 1 000 000 TPM`

Applied in AIService.synthesize using actual token counts from provider response.

Both raise RateLimitExceeded → tenacity retries with exponential backoff

Cache Design & Pydantic Models

src/services/cache.py · src/storage/cache_store.py · src/models.py

TTL In-Memory Cache

```
Key:   (source_name, canonical_query)
```

```
Value: (timestamp: float, list[Source])
```

- Lazy TTL eviction on get (default 3600s)
- --no-cache bypasses both read AND write
- CacheBackend ABC → swap Redis/JSON easily
- Canonical: lower + strip + whitespace-norm
- Ephemeral: wiped on restart, per-process

Pydantic v2 Models (models.py)

ResearchRequest

```
question: str  (min=1, max=500)
sources: list[SourceFilter] | None
no_cache: bool = False
```

ResearchResult

```
question / answer: str
citations: list[CitationRecord]
sources_used / sources_failed
cached: bool · elapsed_seconds: float
```

CitationRecord

```
index: int · title · url · origin
```

Benchmark — Sequential vs Concurrent

scripts/benchmark.py · N=5 questions × 3 sources · Semaphore limit = 5

Offline Simulation

`python scripts/benchmark.py --n 5 --offline`

3.170s	1.311s
Sequential · 1.0×	**Concurrent · 2.42×**

Simulated provider latencies

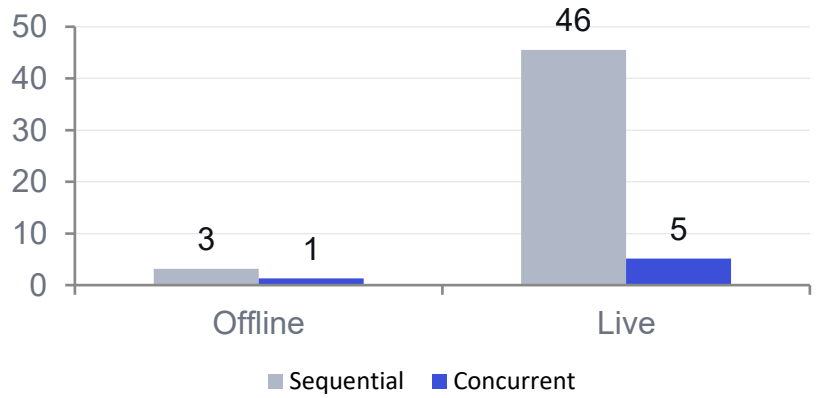
Wikipedia	0.15 s
arXiv	0.20 s
Web search	0.18 s

Wall-clock ≈ max(wiki, arXiv, web) not sum

Live Network (Tavily)

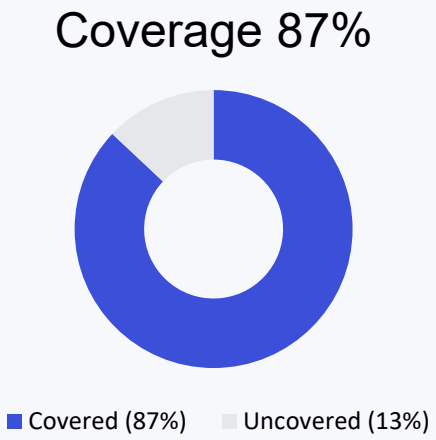
`python scripts/benchmark.py --n 5`

45.513s	5.158s
Sequential · 1.0×	**Concurrent · 8.82×**



Testing Strategy

127 tests · 87% coverage · fully offline · all HTTP mocked via respx / pytest-httpx



127	87%	0	✓
Tests	Coverage	Net calls	Smoke

`conftest.py`

FakeLLM · FakeWebSearch
no_real_ai · clear_cache (autouse)

smoke	`test_ai_smoke.py`	*PROVIDED contract — must always pass*
cache	`test_cache.py`	*TTL, canonical, expiry, clear*
svc	`test_service.py`	*AIService retry, timeout, logging*
db	`test_storage.py`	*repository save / retrieve / error*
db	`test_repository.py`	*PostgreSQL repository (mocked)*
e2e	`test_researcher.py`	*End-to-end pipeline (fully offline)*
async	`test_concurrency.py`	*gather, semaphore, graceful degradation*
cli	`test_cli.py`	*Click CliRunner: ask, history, JSON*
model	`test_models.py`	*Pydantic validation edge cases*
http	`test_http_mocking.py`	*respx HTTP mocks, retry on conn-error*
web	`test_web_provider.py`	*DDG / Tavily / Serper adapters*
log	`test_logging_config.py`	*Logging setup, idempotent, level*

CLI & Streamlit Web UI

src/cli.py (Click) · streamlit_app.py · researcher/ package

CLI · python -m researcher ask / history

```
# All three sources (default)
python -m researcher ask 'What is quantum entanglement?'

# Specific sources, bypass cache
python -m researcher ask 'CRISPR' \
    --sources wiki,arxiv --no-cache

# Machine-readable JSON for piping
python -m researcher ask 'AI safety' --json-output

# History (requires DATABASE_URL)
python -m researcher history --limit 10

# Verbose logging
python -m researcher --log-level DEBUG ask 'AI safety'
```

Streamlit Web UI (bonus)

streamlit run streamlit_app.py → http://localhost:8501

- Research question input with source selector
- Cited Markdown answers with numbered refs
- Session history — persisted via PostgreSQL
- --no-cache toggle for always-fresh results
- Docker single-container or full Compose stack

`docker compose up --build → app + PostgreSQL at :8501`

Database & Storage Layer

src/storage/repository.py · psycopg3 async · PostgreSQL 14+

SQL schema · auto-created on init

```sql
CREATE TABLE IF NOT EXISTS research_sessions (
    id              SERIAL PRIMARY KEY,
    question        TEXT NOT NULL,
    answer          TEXT NOT NULL,
    citations_json  JSONB NOT NULL DEFAULT '[]',
    sources_failed  JSONB NOT NULL DEFAULT '[]',
    cached          BOOLEAN NOT NULL DEFAULT FALSE,
    elapsed_seconds FLOAT NOT NULL DEFAULT 0.0,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_research_sessions_created_at
    ON research_sessions (created_at DESC);
```

```
DATABASE_URL=postgresql://USER:PASS@HOST:5432/DBNAME  (omit → history silently skipped)
```

repository.py features

- AsyncConnectionPool — max_size=5 (env-cfg)
- CREATE TABLE IF NOT EXISTS on first call
- save_session() — full ResearchResult as JSONB
- get_sessions(limit) → list[dict]
- Graceful: warns + continues if DB absent
- PostgreSQL-only: JSONB, SERIAL, TIMESTAMPTZ
- SQLite not supported (psycopg3 %s placeholders)

Project Structure

github.com/abdurahmanligulnisa/research-assistant · tag v1.0-final

```
research-assistant/
  ├── ai/              ← PROVIDED — DO NOT MODIFY
  │   ├── providers/   (Anthropic / OpenAI / Gemini)
  │   ├── schemas.py   Source, Citation, AnswerWithCitations
  │   ├── sources.py   fetch_wikipedia / fetch_arxiv / fetch_web
  │   └── synthesizer.py  synthesize()
  ├── src/             ← student SE layer
  │   ├── config.py    pydantic-settings
  │   ├── models.py    Pydantic models (Request / Result)
  │   ├── logging_config.py  structured logging
  │   ├── services/    ai_service · cache · rate_limiter · web_provider
  │   ├── core/        researcher.py  (full pipeline)
  │   ├── concurrency/ orchestrator.py
  │   ├── storage/     repository.py + cache_store.py
  │   └── cli.py       Click: ask / history
  ├── researcher/  __main__.py  (python -m researcher entry point)
  ├── tests/       13 test files — 127 tests, 87% coverage
  ├── scripts/     demo.py · bench.py · benchmark.py
  ├── artefacts/   bench_results · coverage · pytest_output
  ├── streamlit_app.py   ← Web UI (bonus +1)
  ├── Dockerfile         ← multi-stage (bonus +1)
  ├── docker-compose.yml ← app + PostgreSQL
  └── .github/workflows/ci.yml  ← GitHub Actions (bonus +2)
```

Bonus Features Deep-Dive

Multi-stage Docker (+1 pt) · GitHub Actions CI (+2 pts) · Streamlit UI (+1 pt) · Dual Rate Limiters

Multi-Stage Dockerfile (+1 pt)

STAGE 1 — Builder *python:3.12-slim*

```
pip install -r requirements.txt
into isolated /venv
```

STAGE 2 — Runtime *python:3.12-slim (clean)*

```
COPY --from=builder /venv /venv
No build tools · No pip cache
```

- Runs as non-root user (appuser) — container security
- ~220 MB final image — no build tools or pip cache
- linux/amd64 platform — consistent CI + production builds

Dual Rate Limiters (rate_limiter.py)

① TokenBucketRateLimiter

Controls requests/second per source — auto-applied before every source fetch

10 req/s	2 req/s	1 req/s
Wikipedia	arXiv	Web search

② LlmTpmLimiter (NEW)

Controls LLM tokens-per-minute — auto-applied in AIService.synthesize

40 000 TPM	200 000 TPM	1 000 000 TPM
Claude Sonnet (free tier)	GPT-4o mini	Gemini 2.0 Flash

Both raise RateLimitExceeded → tenacity retries with exponential backoff

Known Limitations

Documented in README §14 and report/report.pdf §8

Ephemeral Cache

cache

In-process dict wiped on restart. Each worker has its own private cache. Fix: replace _store with Redis or filesystem-JSON via CacheBackend ABC.

PostgreSQL History Only

storage

History persisted only if DATABASE_URL is set. Absent or unreachable DB → silently skipped with a WARNING log. SQLite unsupported (psycopg3 syntax).

Bounded Connection Pool

db

AsyncConnectionPool caps at max_size=5 (env-configurable). Heavy concurrent load queues inside the pool rather than opening new connections.

Rate Limiter Is Opt-In

api

TokenBucketRateLimiter (wiki 10/s, arXiv 2/s, web 1/s) exists in rate_limiter.py but is not auto-wired into AIService. Callers must opt in.

DuckDuckGo Rate-Limited

search

Burst traffic can return empty results or raise RatelimitException. Use WEB_SEARCH_PROVIDER=tavily (or serper) for production reliability.

No Cross-Process Sharing

scale

Multiple load-balanced workers each have an isolated cache island. Redis backend swap is straightforward via the CacheBackend ABC.

What We Built

Team ExperimentMice · AI-ENG-110 · May 2026

- Full SE layer: config · cache · concurrency · retries · CLI · storage
- 127 tests, 87% coverage, zero network hits in CI — all mocked
- Multi-stage Dockerfile + docker-compose.yml (app + PostgreSQL)
- GitHub Actions CI workflow — tagged v1.0-final
- 2.42× offline · 8.82× live-network speedup (Tavily, N=5)
- Streamlit Web UI — question input, citations, session history

87%	Test Coverage
127	Tests Passing
2.42×	Offline Speedup
8.82×	Live-Net Speedup
+7 pts	Bonus (Docker+CI+UI+Token-bucket rate limitier)